SHORT PROJECT REPORT

GRIDLEDGER

Electricity Consumption & Billing Management System

Full Stack Development Laboratory Project
Using React.js and MongoDB

Submitted by: Nayana N S

Register Number(s): 24BIT074

Department: Information Technology

Institution: Kumaraguru College of Technology

Academic Year: 2024-2028

1. ABSTRACT

GridLedger is a web-based electricity consumption and billing management system for an electricity service provider. It brings consumer records, meter registration, monthly meter readings, tariff-based billing, payment tracking and customer complaints into one platform. The frontend is developed with React.js and Tailwind CSS, the backend services with Node.js and Express.js, and all data is stored in MongoDB Atlas through Mongoose. Access is controlled by JWT authentication and three server-enforced roles: Admin, Meter Reader and Consumer. Meter readings can be entered manually or imported from CSV files; every row is validated and rejected rows are reported with the reason. Units consumed are calculated from the readings, and bills are generated from a configurable tariff (rate per unit, fixed charge, tax, due days and late fee). The system is a simulation: it does not connect to a real electricity board or payment gateway.

2. INTRODUCTION

An electricity service provider serves hundreds of consumers. Each month, meters must be read, consumption calculated, bills prepared and payments followed up. Doing this on paper or in separate spreadsheets is slow and error-prone: readings may be entered wrongly, the same reading may be entered twice, and it is hard to see which bills are unpaid.

GridLedger addresses this by providing a single web application with role-based access. Administrators manage consumers, meters, tariffs, bills and reports; meter readers enter readings only; and consumers log in to view their own consumption, bills, payment status and complaints. Payments are simulated and the application is intended for learning and demonstration.

3. OBJECTIVES

- To develop a user-friendly full-stack web application for electricity billing management.
- To maintain consumer, meter, reading and bill records in MongoDB.
- To validate meter readings (manual and CSV) and calculate consumption automatically.
- To generate bills from a configurable tariff instead of hard-coded rates.
- To restrict each user to the pages and data allowed for their role, using JWT and server-side checks.
- To give administrators dashboards and downloadable reports, and consumers a view of their own billing history.
- To demonstrate full-stack development using React.js, Node.js, Express.js and MongoDB.

4. TECHNOLOGIES USED

Technology	Purpose
React.js	Frontend user interface (pages, forms, tables)
Tailwind CSS	Consistent responsive styling
React Router	Page navigation and role-based route guards
Axios	API requests from the frontend
Recharts	Dashboard and consumption charts
jsPDF	Bill PDF download
Node.js	Backend runtime environment
Express.js	Server and REST API development
MongoDB Atlas	Cloud database storage
Mongoose	Database schemas, validation and relationships
bcrypt	Password hashing
jsonwebtoken (JWT)	Stateless authentication and role claims
Multer and csv-parse	CSV file upload and parsing
VS Code, Vite, nodemon	Development environment and tooling

5. SYSTEM DESIGN

The application follows a three-layer architecture. React.js forms the presentation layer. Node.js and Express.js form the application layer, handling authentication, validation, billing logic and API requests. MongoDB Atlas stores the data and is accessed through Mongoose. The frontend communicates with the backend through HTTP requests carrying a JWT token.

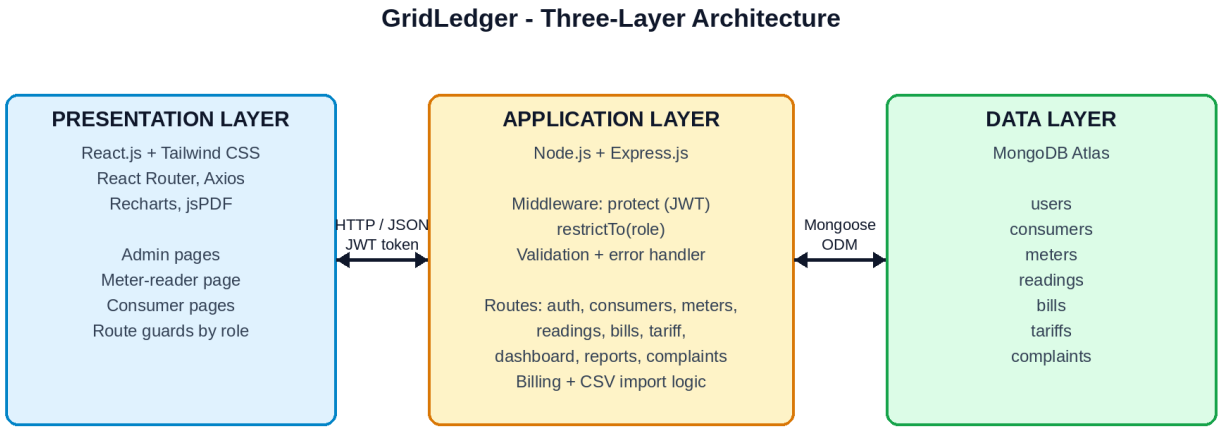


Figure 1: System architecture of GridLedger

5.1 User roles

Role	Access
Admin	Full control: dashboard, consumers, meters, readings and CSV import, tariff, bill generation, payment status, reports and complaint replies.
Meter Reader	Readings page only: manual entry, CSV import and deletion of a recent mistaken entry. Cannot see billing, consumers or reports, and cannot generate bills.
Consumer	Own dashboard, consumption chart, bills, bill PDF, simulated payment and complaints. Cannot see other consumers' data.

5.2 Billing logic

Units consumed = Current reading - Previous reading. Energy charge = Units x Rate per unit. Bill amount = (Energy charge + Fixed charge) + Tax on that total. For example, with a rate of Rs. 5, a fixed charge of Rs. 50 and 5% tax, 150 units give (150 x 5 + 50) = 800, plus 5% tax = Rs. 840. The due date is the bill date plus the configured due days. If a bill is still pending after its due date, a late fee (a configurable percentage of the bill amount) is added and stored when the bill is paid.

5.3 Reading validation rules

- Current reading must not be lower than the previous reading.
- Only one reading per meter per billing month (duplicates are rejected).
- The meter must exist and must belong to the stated consumer.
- Missing, non-numeric or invalid-date values are rejected.
- A reading can be deleted only before it is billed, and only if it is the latest reading of that meter.

6. MODULES

Authentication Module: Login with password hashing (bcrypt) and JWT. The token carries the user role; the server checks it on every protected API and the frontend redirects users away from pages not allowed for their role.

Dashboard Module: Shows total consumers, registered meters, bills generated, revenue collected, pending payments and a monthly consumption chart, all calculated from database records. Consumers see a personal dashboard.

Consumer Management: Admin can add, view, update and delete consumers with search; each consumer also gets a login account.

Meter Management: Register meters, assign them to consumers, record installation date and status, and view reading history.

Meter Reading Management: Manual entry (previous reading is filled automatically from the last reading) and CSV import with validation. Rejected rows are listed with reasons.

Billing Module: Configurable tariff, bill generation from un-billed readings, due dates, late fee on overdue bills and bill PDF download.

Payment Tracking: Admin records counter payments (cash, UPI, card, net banking) and consumers can pay their own bills through a simulated Pay Now option. No real payment gateway is used.

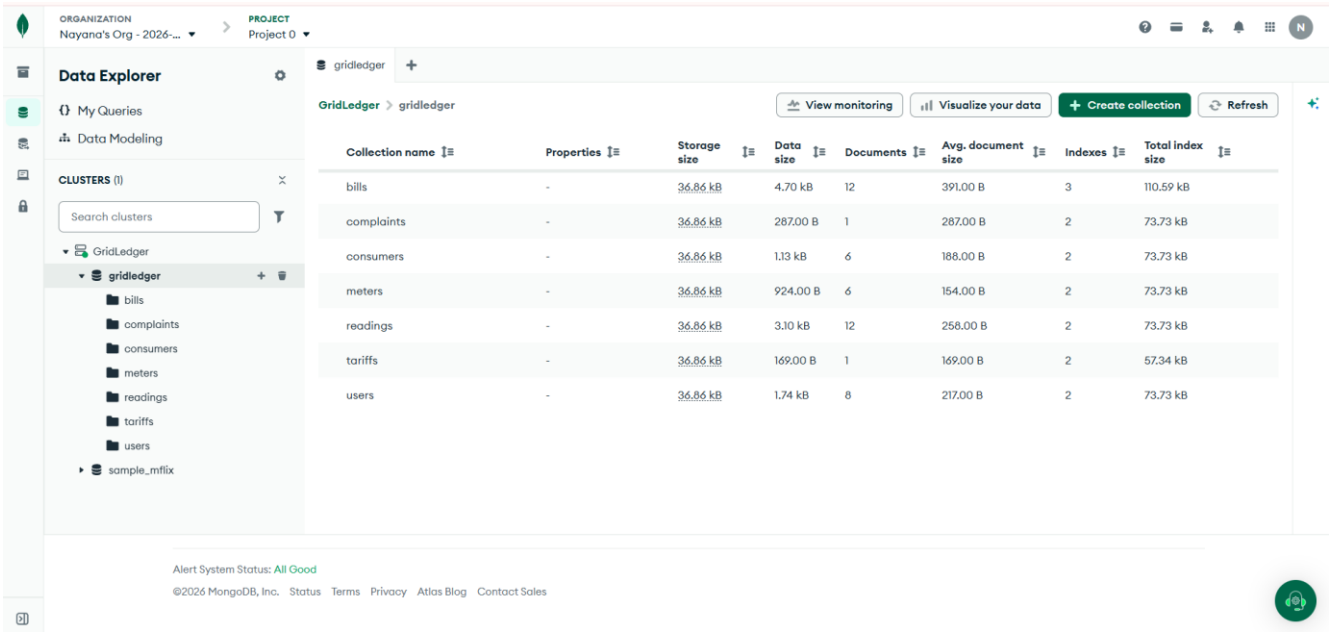
Reports and Analytics: Revenue summary (billed, collected, outstanding), consumer-wise consumption and pending bills, each with CSV export.

Complaints Module: Consumers raise complaints by category; the admin replies and changes the status (open, in progress, resolved).

7. DATABASE DESIGN

MongoDB stores the data in seven collections. Documents are linked with ObjectId references, and unique constraints are applied to identifiers such as email, consumer ID, meter ID and bill number.

Collection	Main fields
users	name, email (unique), passwordHash, role (admin / meter_reader / consumer)
consumers	consumerId (unique), name, address, phone, user (reference)
meters	meterId (unique), consumer (reference), installationDate, status
readings	meterId, consumerId, period, readingDate, previousReading, currentReading, units, source (manual / csv)
bills	billNo (unique), consumer, reading (unique reference), period, units, ratePerUnit, energyCharge, fixedCharge, taxAmount, amount, lateFeePercent, lateFee, dueDate, status, paidDate, paymentMethod
tariffs	ratePerUnit, fixedCharge, taxPercent, dueDays, lateFeePercent (single configurable document)
complaints	complaintNo (unique), consumer, category, subject, description, status, adminReply, resolvedAt

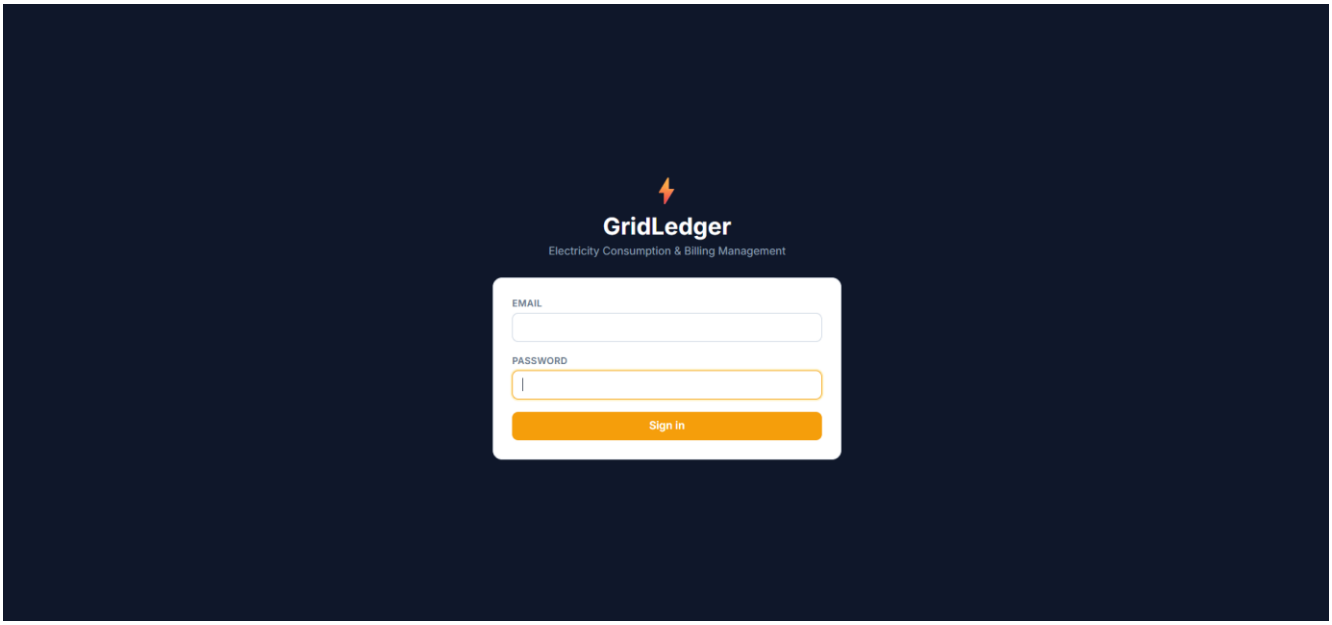


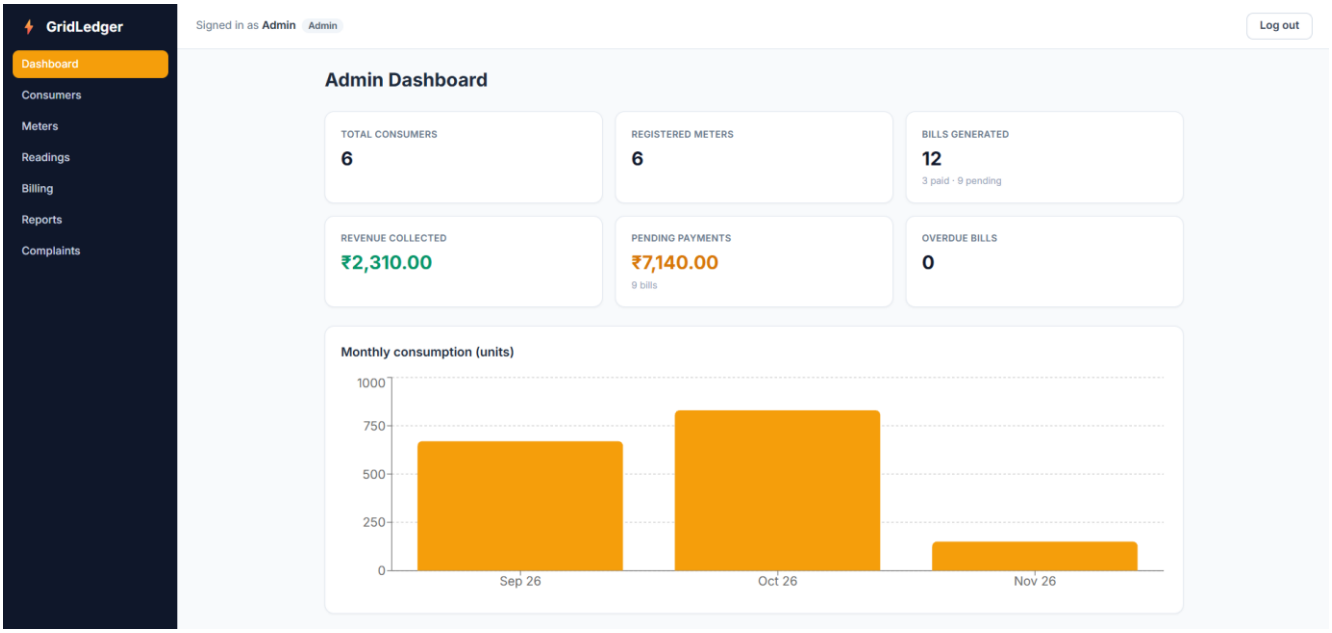
8. IMPLEMENTATION

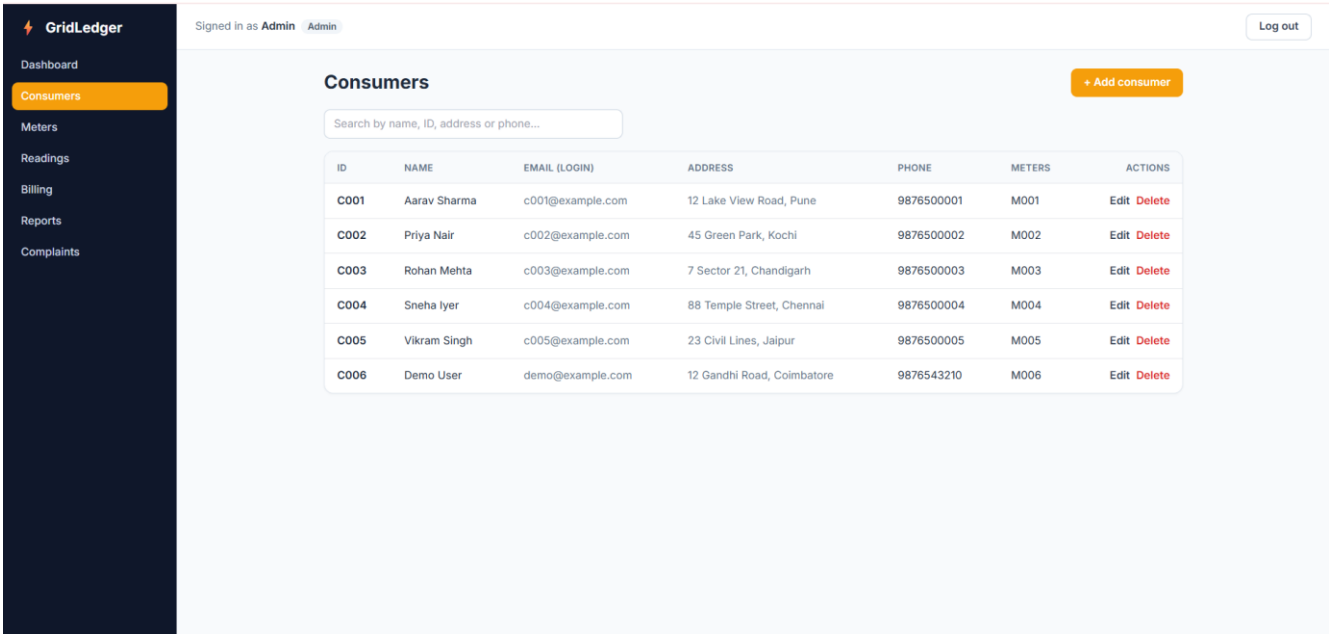
The frontend is built from reusable React components (sidebar, protected route, stat cards, badges, modal) and pages for each module. It communicates with the backend through Axios. Express.js routes receive requests, validate input and perform database operations using Mongoose. Authorization is enforced by the protect middleware, which verifies the JWT, and the restrictTo middleware, which allows only the listed roles on each route.

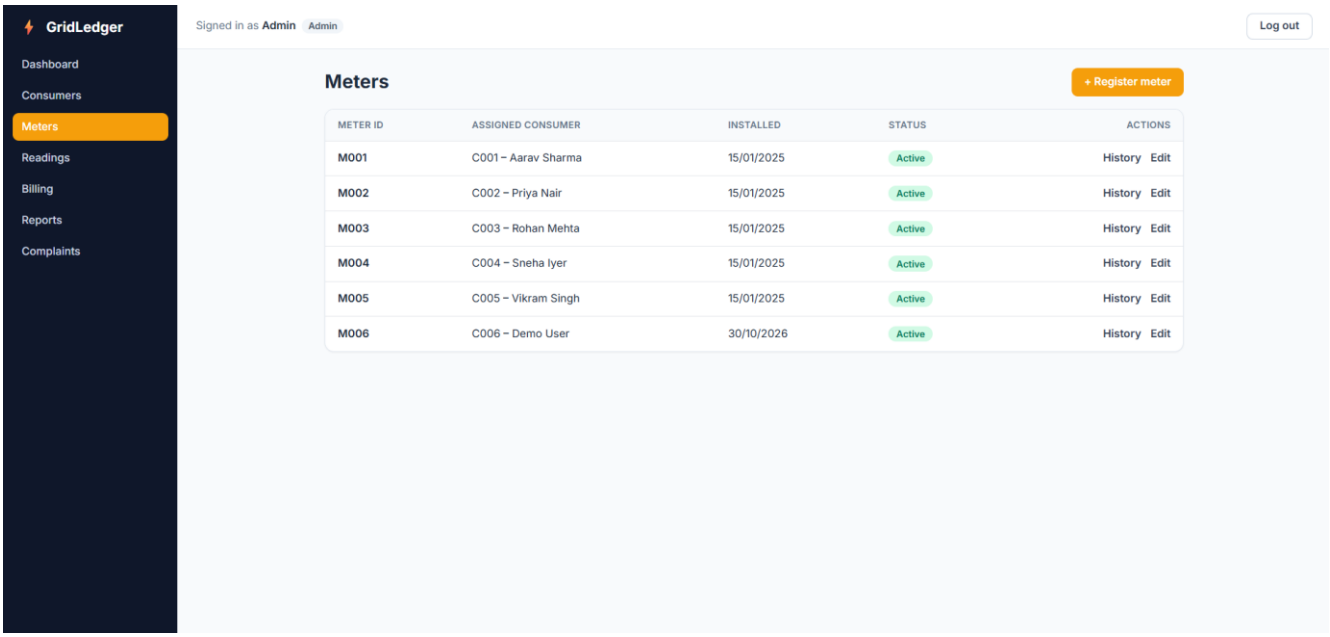
API endpoint	Purpose
POST /api/auth/login	Login; returns JWT and user role
/api/consumers, /api/meters	Consumer and meter management (admin)
GET, POST /api/readings	List and enter readings (admin, meter reader)
POST /api/readings/upload	CSV import with row-wise validation
DELETE /api/readings/:id	Undo a mistaken, unbilled latest reading
POST /api/bills/generate	Generate bills for un-billed readings (admin)
PATCH /api/bills/:id/payment	Admin records or undoes a payment
PATCH /api/bills/:id/pay	Consumer pays own bill (simulated)
GET, PUT /api/tariff	View and update tariff (update: admin)
GET /api/dashboard, /api/reports	Dashboard figures and reports
/api/complaints	Raise, list and reply to complaints

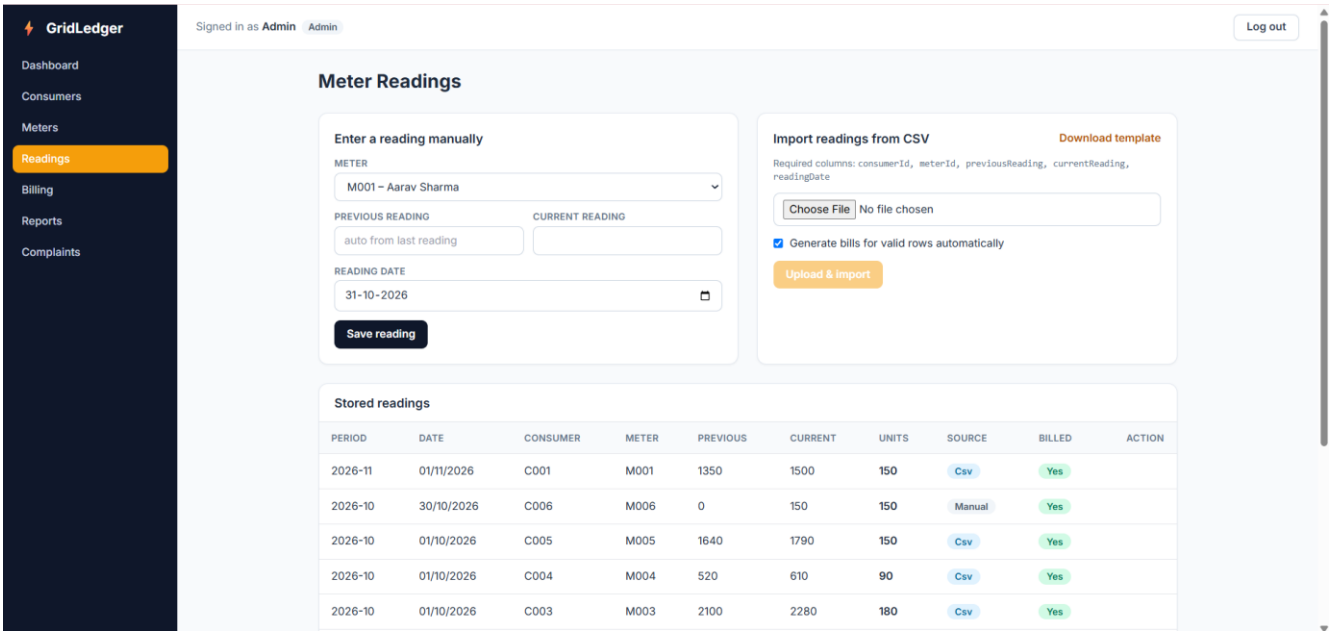
CSV import reads each row, checks the required columns (consumerId, meterId, previousReading, currentReading, readingDate), validates the values, saves valid readings and returns the list of rejected rows with reasons. Sample files for September, October and an error file were used to demonstrate this.

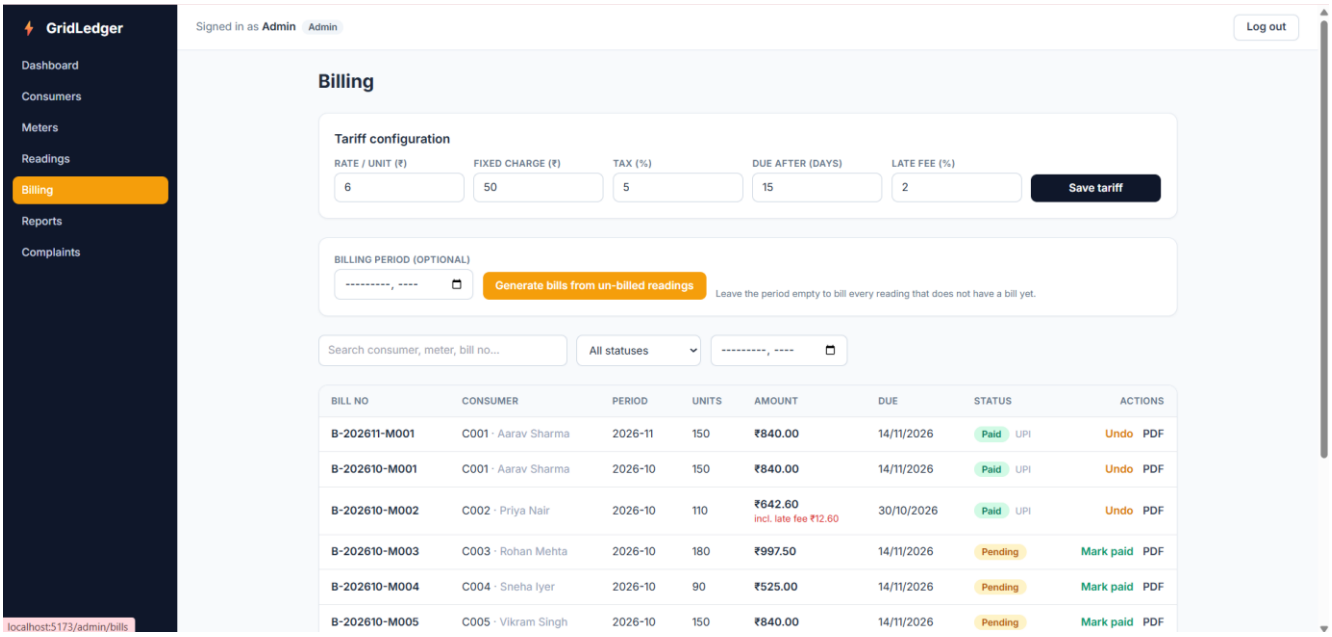

GridLedger
Electricity Consumption & Billing Management
EMAIL
PASSWORD
Sign in

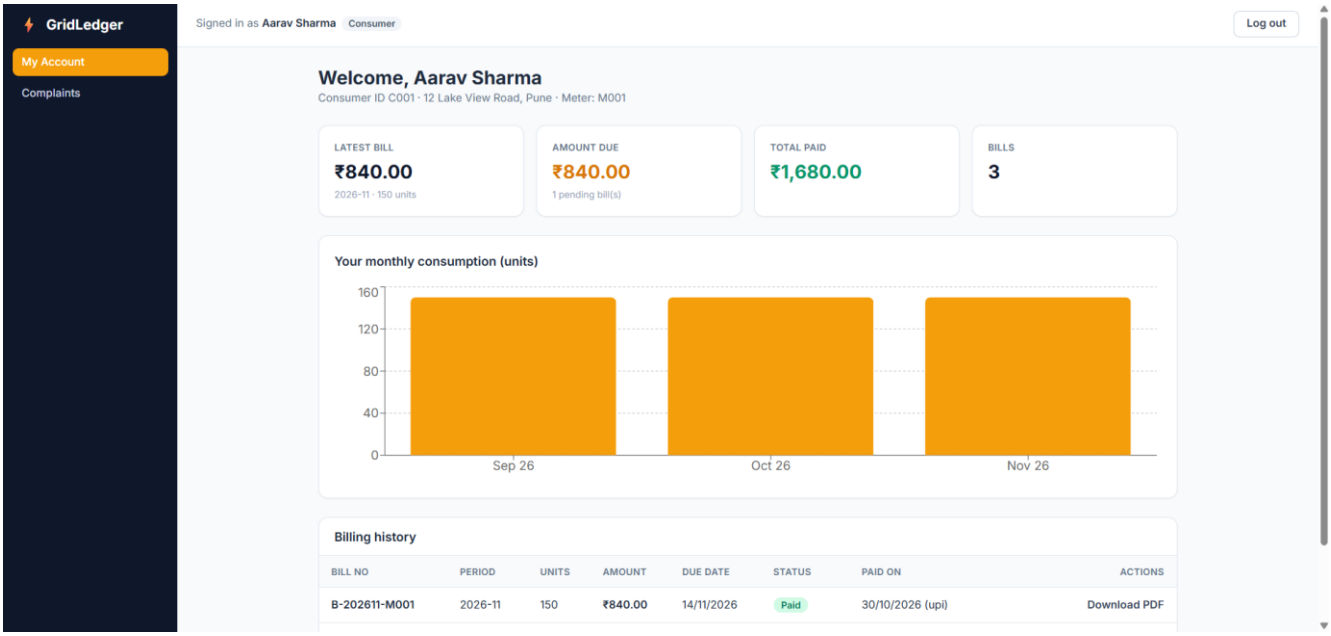

GridLedger
Signed in as Admin
Admin
Log out
Dashboard
Consumers
Meters
Readings
Billing
Reports
Complaints
Admin Dashboard
TOTAL CONSUMERS
6
REGISTERED METERS
6
BILLS GENERATED
12
3 paid · 9 pending
REVENUE COLLECTED
₹2,310.00
PENDING PAYMENTS
₹7,140.00
9 bills
OVERDUE BILLS
0
Monthly consumption (units)
1000
750
500
250
0
Sep 26
Oct 26
Nov 26

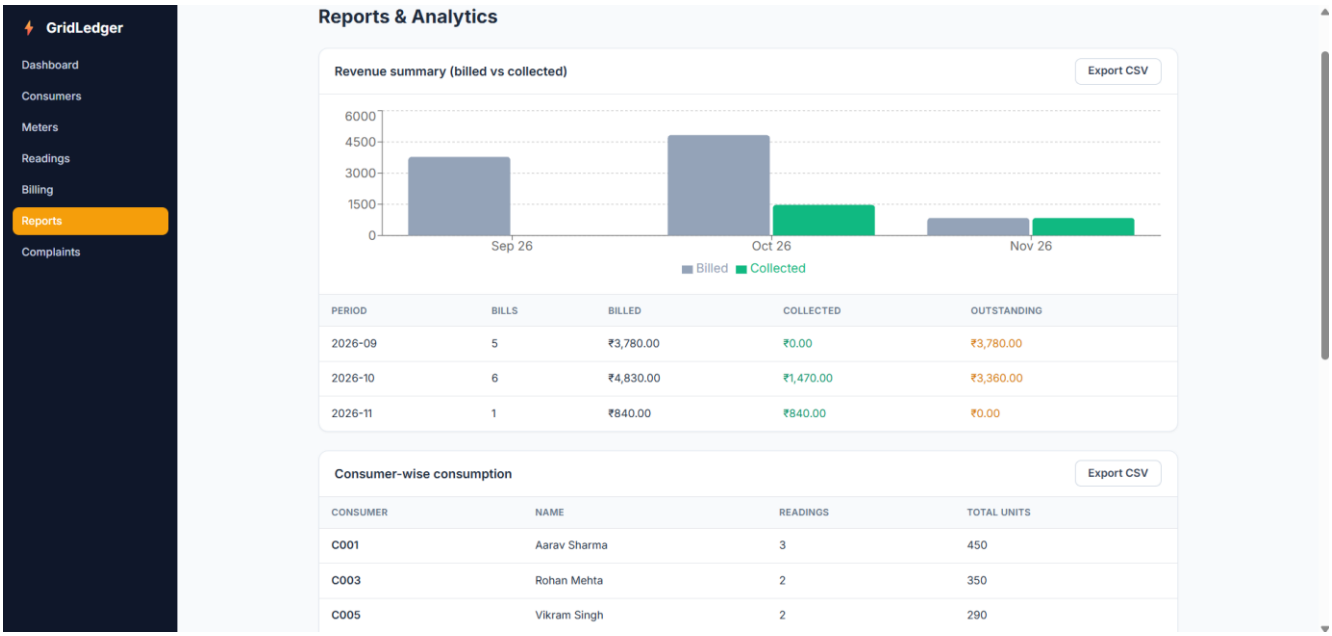

GridLedger
Dashboard
Consumers
Meters
Readings
Billing
Reports
Complaints
Signed in as Admin Admin
Log out
Consumers
+ Add consumer
Search by name, ID, address or phone...
ID NAME EMAIL (LOGIN) ADDRESS PHONE METERS ACTIONS
C001 Aarav Sharma c001@example.com 12 Lake View Road, Pune 9876500001 M001 Edit Delete
C002 Priya Nair c002@example.com 45 Green Park, Kochi 9876500002 M002 Edit Delete
C003 Rohan Mehta c003@example.com 7 Sector 21, Chandigarh 9876500003 M003 Edit Delete
C004 Sneha Iyer c004@example.com 88 Temple Street, Chennai 9876500004 M004 Edit Delete
C005 Vikram Singh c005@example.com 23 Civil Lines, Jaipur 9876500005 M005 Edit Delete
C006 Demo User demo@example.com 12 Gandhi Road, Coimbatore 9876543210 M006 Edit Delete

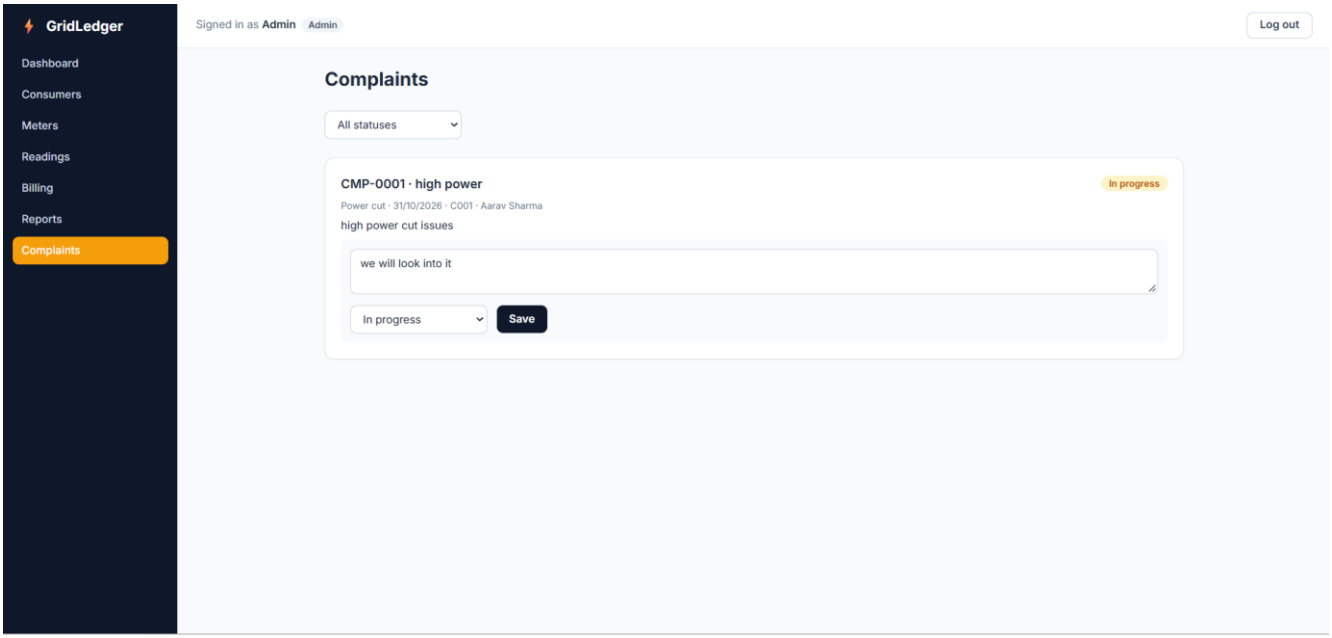

GridLedger
Dashboard
Consumers
Meters
Readings
Billing
Reports
Complaints
Signed in as Admin Admin
Log out
Meters
+ Register meter
METER ID ASSIGNED CONSUMER INSTALLED STATUS ACTIONS
M001 C001 – Aarav Sharma 15/01/2025 Active History Edit
M002 C002 – Priya Nair 15/01/2025 Active History Edit
M003 C003 – Rohan Mehta 15/01/2025 Active History Edit
M004 C004 – Sneha Iyer 15/01/2025 Active History Edit
M005 C005 – Vikram Singh 15/01/2025 Active History Edit
M006 C006 – Demo User 30/10/2026 Active History Edit

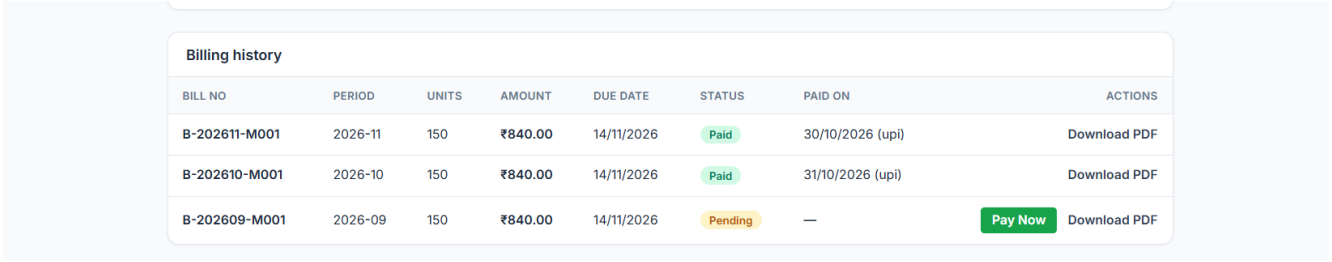

GridLedger
Signed in as Admin Admin
Log out
Dashboard
Consumers
Meters
Readings
Billing
Reports
Complaints
Meter Readings
Enter a reading manually
METER
M001 – Aarav Sharma
PREVIOUS READING
auto from last reading
CURRENT READING
READING DATE
31-10-2026
Save reading
Import readings from CSV
Download template
Required columns: consumerId, meterId, previousReading, currentReading, readingDate
Choose File No file chosen
Generate bills for valid rows automatically
Upload & import
Stored readings
PERIOD DATE CONSUMER METER PREVIOUS CURRENT UNITS SOURCE BILLED ACTION
2026-11 01/11/2026 C001 M001 1350 1500 150 Csv Yes
2026-10 30/10/2026 C006 M006 0 150 150 Manual Yes
2026-10 01/10/2026 C005 M005 1640 1790 150 Csv Yes
2026-10 01/10/2026 C004 M004 520 610 90 Csv Yes
2026-10 01/10/2026 C003 M003 2100 2280 180 Csv Yes

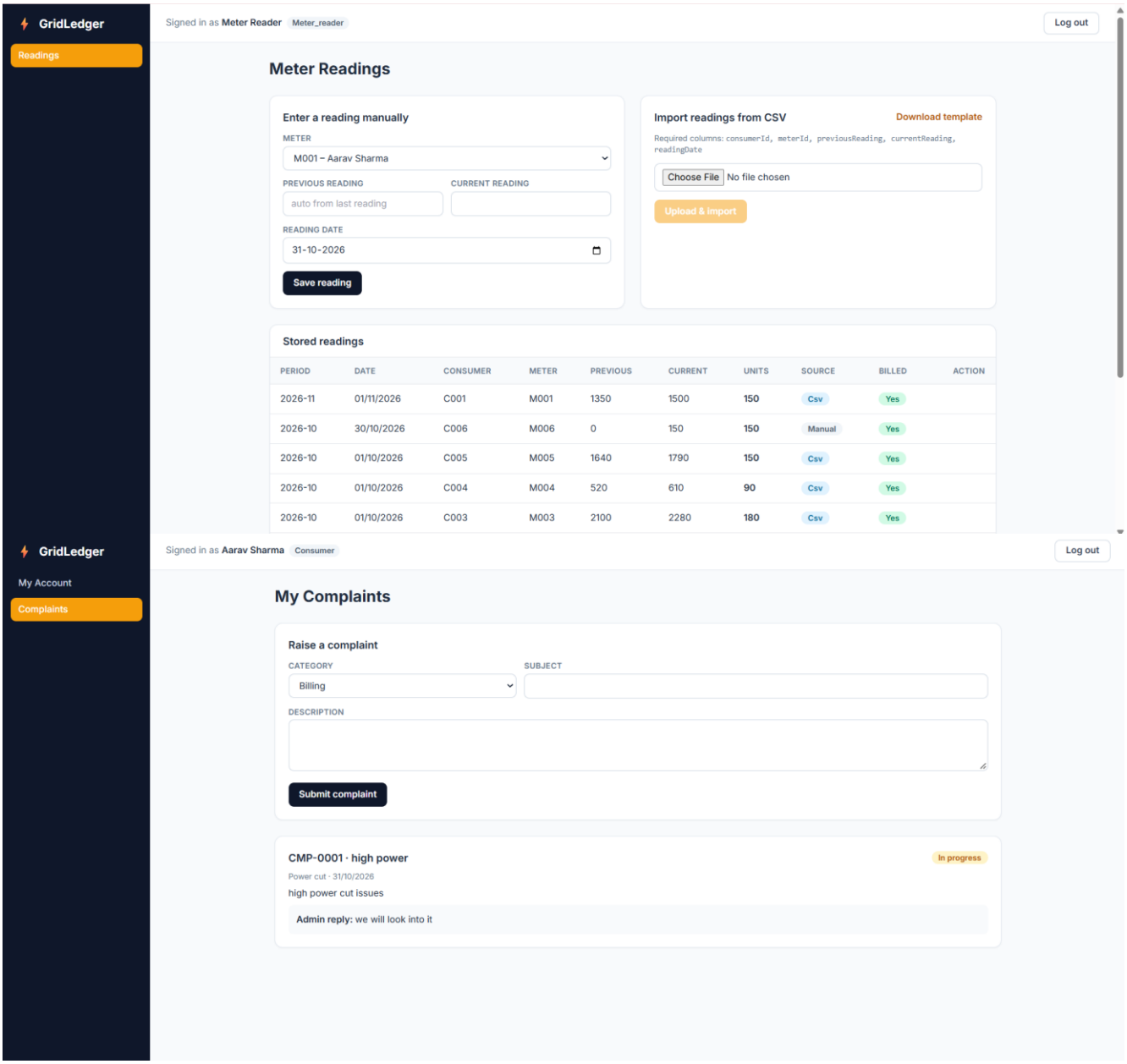

GridLedger
Signed in as Admin Admin
Log out
Dashboard
Consumers
Meters
Readings
Billing
Reports
Complaints
Billing
Tariff configuration
RATE / UNIT (₹)
6
FIXED CHARGE (₹)
50
TAX (%)
5
DUE AFTER (DAYS)
15
LATE FEE (%)
2
Save tariff
BILLING PERIOD (OPTIONAL)
Generate bills from un-billed readings
Leave the period empty to bill every reading that does not have a bill yet.
Search consumer, meter, bill no...
All statuses
BILL NO CONSUMER PERIOD UNITS AMOUNT DUE STATUS ACTIONS
B-202611-M001 C001 · Aarav Sharma 2026-11 150 ₹840.00 14/11/2026 Paid UPI Undo PDF
B-202610-M001 C001 · Aarav Sharma 2026-10 150 ₹840.00 14/11/2026 Paid UPI Undo PDF
B-202610-M002 C002 · Priya Nair 2026-10 110 ₹642.60 incl. late fee ₹12.60 30/10/2026 Paid UPI Undo PDF
B-202610-M003 C003 · Rohan Mehta 2026-10 180 ₹997.50 14/11/2026 Pending Mark paid PDF
B-202610-M004 C004 · Sneha Iyer 2026-10 90 ₹525.00 14/11/2026 Pending Mark paid PDF
B-202610-M005 C005 · Vikram Singh 2026-10 150 ₹840.00 14/11/2026 Pending Mark paid PDF
localhost:5173/admin/bills


GridLedger
My Account
Complaints
Signed in as Aarav Sharma Consumer
Log out
Welcome, Aarav Sharma
Consumer ID C001 · 12 Lake View Road, Pune · Meter: M001
LATEST BILL
₹840.00
2026-11 · 150 units
AMOUNT DUE
₹840.00
1 pending bill(s)
TOTAL PAID
₹1,680.00
BILLS
3
Your monthly consumption (units)
160
120
80
40
0
Sep 26
Oct 26
Nov 26
Billing history
BILL NO
PERIOD
UNITS
AMOUNT
DUE DATE
STATUS
PAID ON
ACTIONS
B-202611-M001
2026-11
150
₹840.00
14/11/2026
Paid
30/10/2026 (upi)
Download PDF


GridLedger
Dashboard
Consumers
Meters
Readings
Billing
Reports
Complaints
Reports & Analytics
Revenue summary (billed vs collected)
Export CSV
6000
4500
3000
1500
0
Sep 26
Oct 26
Nov 26
Billed
Collected
PERIOD
BILLS
BILLED
COLLECTED
OUTSTANDING
2026-09
5
₹3,780.00
₹0.00
₹3,780.00
2026-10
6
₹4,830.00
₹1,470.00
₹3,360.00
2026-11
1
₹840.00
₹840.00
₹0.00
Consumer-wise consumption
Export CSV
CONSUMER
NAME
READINGS
TOTAL UNITS
C001
Aarav Sharma
3
450
C003
Rohan Mehta
2
350
C005
Vikram Singh
2
290


GridLedger
Signed in as Admin Admin
Log out
Dashboard
Consumers
Meters
Readings
Billing
Reports
Complaints
Complaints
All statuses
CMP-0001 · high power
In progress
Power cut · 31/10/2026 · C001 · Aarav Sharma
high power cut issues
we will look into it
In progress
Save


Billing history
BILL NO
PERIOD
UNITS
AMOUNT
DUE DATE
STATUS
PAID ON
ACTIONS
B-202611-M001
2026-11
150
₹840.00
14/11/2026
Paid
30/10/2026 (upi)
Download PDF
B-202610-M001
2026-10
150
₹840.00
14/11/2026
Paid
31/10/2026 (upi)
Download PDF
B-202609-M001
2026-09
150
₹840.00
14/11/2026
Pending
—
Pay Now
Download PDF



9. TESTING AND RESULTS

The application was tested manually by running the complete workflow with data stored in MongoDB Atlas. The tests cover the main operations of each module, input validation, role-based access and payment handling.

Test	Expected Result	Status/output
Admin and consumer login	User is signed in and sent to the page for their role	Passed
Add consumer and register meter	Records are saved and listed	Passed
CSV import (September and October files)	5 rows accepted, 5 bills generated for each file	Passed
CSV import with errors	1 row accepted, 8 rejected with reasons	Passed
Units calculation (1500 to 1650)	Units shown as 150	Passed
Tariff change and new bill	150 units at Rs. 6, Rs. 50 fixed, 5% tax gives Rs. 997.50	Passed
Admin marks bill paid	Status changes to paid and stays after refresh	Passed

Test	Expected Result	Status/output
Consumer Pay Now	Bill shows paid in admin Billing page; revenue increases	Passed
Dashboard and reports	Figures match database records	Passed
Late fee on overdue bill	Rs. 630 bill shows Rs. 642.60 with 2% late fee	Passed
Delete mistaken reading	Allowed before billing; blocked after billing	Passed
Role-based access	Reader and consumer redirected from admin pages; consumers see only own bills and complaints	Passed
Complaint raised and replied	Consumer sees admin reply and status	Passed
Database connection	Data is retrieved successfully from MongoDB Atlas	Passed

10. CONCLUSION

GridLedger demonstrates the development of a complete full-stack web application using React.js, Node.js, Express.js and MongoDB. It covers the full billing cycle: registering consumers and meters, entering and validating readings, calculating consumption, generating bills from a configurable tariff, tracking payments and handling complaints, with three roles protected by JWT and server-side authorization. The project helped apply concepts of frontend development, REST API design, authentication, data validation and database integration.

Limitations and future scope

- Payments are simulated; a real payment gateway can be integrated.
- SMS or email notifications for due dates are not included.
- Tariff uses a single rate; slab-based tariffs can be added.
- Live smart-meter (IoT) integration and consumption prediction are possible enhancements.

11. REFERENCES

- React Documentation – https://react.dev/
- Node.js Documentation – https://nodejs.org/docs/
- Express Documentation – https://expressjs.com/
- MongoDB Documentation – https://www.mongodb.com/docs/
- Mongoose Documentation – https://mongoosejs.com/docs/
- Tailwind CSS Documentation – https://tailwindcss.com/docs
- Recharts Documentation – https://recharts.org/
- JSON Web Token Introduction – https://jwt.io/introduction